# Informative document

## What G-CHAT Is

Every organisation has data files that nobody actually queries — CSV exports, Excel reports, transaction dumps — sitting in storage, untouched, because getting anything useful out of them requires a developer or a data analyst to write SQL.

**G-CHAT removes that dependency.**

Point it at your Azure Blob Storage — the folder your files already live in. It reads every file, understands what each one contains, and makes all of them queryable in plain English through a chat interface.

> A manager types: *"Show me the top 10 customers by outstanding balance this quarter."* G-CHAT identifies the right file, writes and runs the query, and returns the table — in under 30 seconds. No SQL. No BI tool. No analyst in the loop.

The people who own the business questions can now ask them directly.

## How It Works

You provide one thing: your **Azure Blob Storage connection string**. That is the only credential we hold. We never touch your application database or connect to your MongoDB.

When a file lands in your storage container, G-CHAT automatically:

1. **Reads a sample of the file** — detects structure, column names, data types, and date ranges to understand what the file is good for.
2. **Converts it to Parquet** — a fast binary format that makes queries run in seconds rather than minutes on large files.
3. **Stores a compact searchable index** — not your data, just metadata — in its own internal database.

From that point, any user with access can query that file in plain English. The AI identifies the right file, writes the SQL, runs it against your Azure storage, and returns the result.

---

## Why LangChain + LangGraph

When a user sends a message, G-CHAT does not simply ask GPT-4 "what is the answer?" and return whatever it says. That approach fails the moment a query returns empty results, the AI picks the wrong file, or a follow-up is needed.

G-CHAT runs a **structured reasoning loop** — the AI examines your files, makes decisions, runs queries, checks results, and recovers from errors before producing an answer.

**LangChain** is the wiring layer. It connects the AI model to real tools — run a SQL query, look up a file schema, search the catalog — with production-grade error handling and retry logic built in. It also keeps us provider-agnostic: switching from Azure OpenAI to any other model is two lines of config.

**LangGraph** manages multi-step reasoning. Here is a real example:

> *User asks: "Who are the top customers by revenue this month?"*

| Step | What Happens |
|---|---|
| 1 | Search the file catalog for the right file |
| 2 | Inspect the schema to understand column names |
| 3 | Run the query |
| 4 | If 0 rows returned — detect why (wrong date range? wrong file?) and run a corrective step automatically |
| 5 | Return the correct answer |

The user sees the right answer. The loop is invisible to them.

For a system where data lives across hundreds of files with varied naming conventions, that recovery loop is not optional — it is the difference between a system that works reliably and one that fails on anything beyond a simple question.

---

## Why PostgreSQL, Not MongoDB

**First, the reassurance:** your MongoDB is completely separate. We never connect to it. We never ask for its credentials. This is an implementation detail entirely on our side — it has no effect on your application or your data.

The only thing we connect to is your Azure Blob Storage container.

G-CHAT needs four things to happen on every single user message:

| Capability | PostgreSQL | MongoDB |
|---|---|---|
| **Semantic search** (find files by meaning) | pgvector — 20–80 ms, built in | Atlas Vector Search — 50–200 ms, paid add-on |
| **Keyword + fuzzy search** (typo tolerance, ranking) | Built in, free | Requires separate paid Atlas Search index |
| **Relational queries** (permissions, file relationships) | Standard JOINs — 5–30 ms | $lookup — 100–500 ms |
| **Transactional writes** (atomic multi-table ingestion) | Native since 1996 | Added in 2018; slower under load |

G-CHAT needs all four simultaneously, on every message. PostgreSQL handles all four from a single database. MongoDB handles one natively and requires separate paid services for the rest.

**Does this affect your data or query speed?** No. Your business data stays in Azure Blob Storage. G-CHAT's PostgreSQL database contains only metadata: file descriptions, column names, user permissions, conversation history.

When a user asks a question:

| Step | What Happens | Time |
|---|---|---|
| 1 | Search PostgreSQL for the right files | 20–50 ms |
| 2 | Run the query via DuckDB on Azure | 1–30 s |
| 3 | GPT-4 generates the answer | 2–5 s |

The speed of your queries is determined by your file sizes and Azure network latency — not by our internal database choice.

---

## Ingestion — What Happens to Every File

When a file is uploaded or detected in your Azure container, G-CHAT runs the following automatically. No manual steps, no configuration required from your side.

| Stage | What Happens | Time |
|---|---|---|
| **1 — Detection** | Detects file type, delimiter, and character encoding. Legacy non-UTF-8 formats handled correctly. | ~30 ms |
| **2 — Excel splitting** | Multi-sheet workbooks are split into individual files. Each sheet is queryable independently. | 100–300 ms per workbook |
| **3 — Schema inference** | Reads 500 rows. Determines column names and data types. Re-ingested files with new columns are processed incrementally. | 200–500 ms |
| **4 — Quality profiling** | For every column: percentage blank, percentage junk values ("N/A", "TBD", "–"). Stored so the AI knows data quality before writing any query. | 50–100 ms |
| **5 — AI description** | GPT-4 reads column names, data types, and sample rows and writes a plain-English description of what the file contains and what questions it can answer. | 2–4 s |
| **6 — Relationship detection** | Column names are compared against every other ingested file. Files sharing join-compatible columns are linked. Used automatically for multi-file queries. | 100–500 ms |
| **7 — Parquet conversion** | File compressed into fast binary format and stored in Azure. A 2 GB CSV that takes minutes to query directly takes under 2 seconds as Parquet. Runs in the background — file is queryable before this finishes. | 1 s – 5 min |

## Time from Upload to Queryable

| File Size | Ready In |
|---|---|
| Under 10 MB | 5–10 seconds |
| 10 MB – 100 MB | 10–25 seconds |
| 100 MB – 1 GB | 30–90 seconds |

Parquet conversion finishes silently in the background. Users are never waiting for it.

## At Scale

If you have a large backlog of existing files in Azure:

- **Phase 1** — All files appear in the file manager immediately. Users can see what exists within minutes of setup.
- **Phase 2** — Background workers process files in parallel. Files become queryable as each one completes. A bad or malformed file does not block others.
- **Ongoing** — New files uploaded to Azure are processed within 5–30 seconds of arrival, with no manual trigger.

---

## Summary

> You provide your Azure Blob Storage connection string.
> G-CHAT makes every file in it queryable in plain English — in under 30 seconds per file — without touching anything else in your infrastructure.

---